
boolean

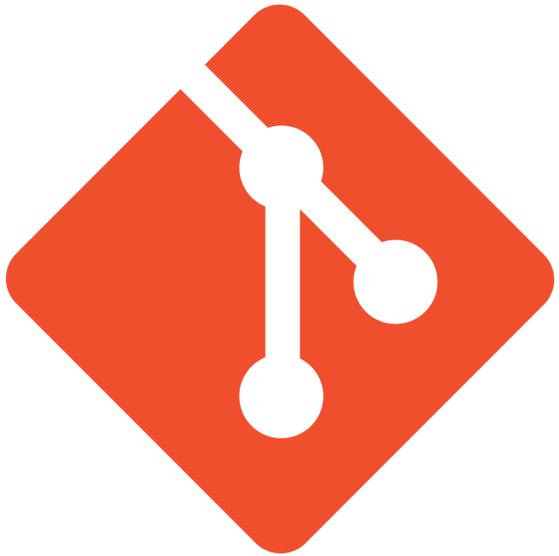


GIT

GIT - Version Control

Tracking delle modifiche e condivisione del codice

Git è un sistema di versionamento, che ci permette di tracciare ogni step di progetto e di avere diverse versioni dello stesso.
Sia sul nostro computer che su piattaforme esterne, attraverso le quali possiamo condividere il nostro progetto con altre persone e quindi anche lavorarci in team in parallelo.



Perché serve?

Per rispondere (tra le altre) alle seguenti situazioni:

- Quale modifica avevo appena fatto?
- Come mi piacerebbe poter tornare a prima quando almeno funzionava tutto!
- Cosa avevo scritto quando quel div era posizionato bene?
- ...e tante altre domande figlie delle frustrazione

Git è uno strumento che, opportunamente utilizzato,
permette di rispondere alle domande sopra e molto di più.

Distributed version control software (DVCS)

- Tiene traccia delle modifiche
- Permette di annullare le modifiche e tornare ad uno "stato" precedente
- Permette di collaborare in più persone sul codice
- Se collaborativo, permette di vedere chi ha fatto quali modifiche

Quando nasce?

Iniziato a sviluppare da Linus Torvalds nel 2005 per aiutare lo sviluppo del Kernel Linux

- Chi è Linus?
- Cos'è Linux?
- Cos'è un Kernel?

I punti di forza

Esistevano già altri VCS ma più primitivi, Git è stato costruito per avere le seguenti caratteristiche:

- Velocità
- Design semplice
- Massimo supporto allo sviluppo non lineare (migliaia di branch paralleli possibili)
- Completamente distribuito (quindi decentralizza le versioni di progetto)
- Massima compatibilità e quindi diffusione
- Capacità di sostenere in modo efficiente grandi progetti come il Kernel Linux (15+ million LOC)

Perché proprio Git?

È lo standard *de facto* del VCS:

- Se lavorerete in una azienda che usa Subversion, devono esserci dei motivi convincenti, altrimenti...RUN!



Ma non è un po' troppo complicato?

Sì è complicato, soprattutto all'inizio e non sembra che ne valga la pena MA...

NON È COSÌ!

Git è l'unico strumento in grado di garantire che:

- il nostro lavoro non venga perso
- ci sia uno stato funzionante sempre recuperabile
- sia possibile condividere / chiedere aiuto sul codice in maniera efficiente

Ma è davvero così importante?

[DVCS is] possibly the biggest advance in software development technology in the [past] ten years

- Joel Spolsky

Trello, Excel, StackOverflow Q&A



GIT - Init

Inizializzazione del repository

Operazione da fare solo all'inizio di ogni progetto.

Da questo momento ogni modifica sarà tracciata
e la tua cartella di progetto diventa un/una **repo**.

🧐 Significa: *"Ok Git, questa è la cartella che dovrai guardare"*.

Repository (o repo):
è l'insieme di tutti i file del progetto e la loro storia

GIT - Staging

Inserimento dei file nell'area di staging

Git controlla i file del repo e segnala quali sono quelli che sono stati modificati, rispetto all’ultimo stato.

È nostro compito selezionare quelli che logicamente fanno parte della stessa modifica inserendoli nell’area di staging.

(Esempio: per aggiungere un footer ho aggiunto il tag footer modificando il file index.html e aggiunto il selettore footer nel file style.css. Index.html e style.css sono entrambi da inserire nell’area di staging per la modifica "aggiunta del footer").

🧐 Significa: *"Ok Git, raggruppa queste modifiche e memorizza temporaneamente questo stato".*

Staging:
è l'area di sosta temporanea comune per tutti i file che concorrono alla stessa modifica

GIT - Commit

Creazione di un commit

Quando tutti i file necessari a fare una modifica sono stati modificati e inseriti nell’area di staging, si può creare un commit.

Git prende tutte le informazioni dei file nell’area di staging e crea uno "stato di progetto" sulla linea temporale.

Questo potrà essere utilizzato anche come punto di recupero. È un commit e sarà possibile tornare allo stato del progetto al momento del commit in qualsiasi momento.

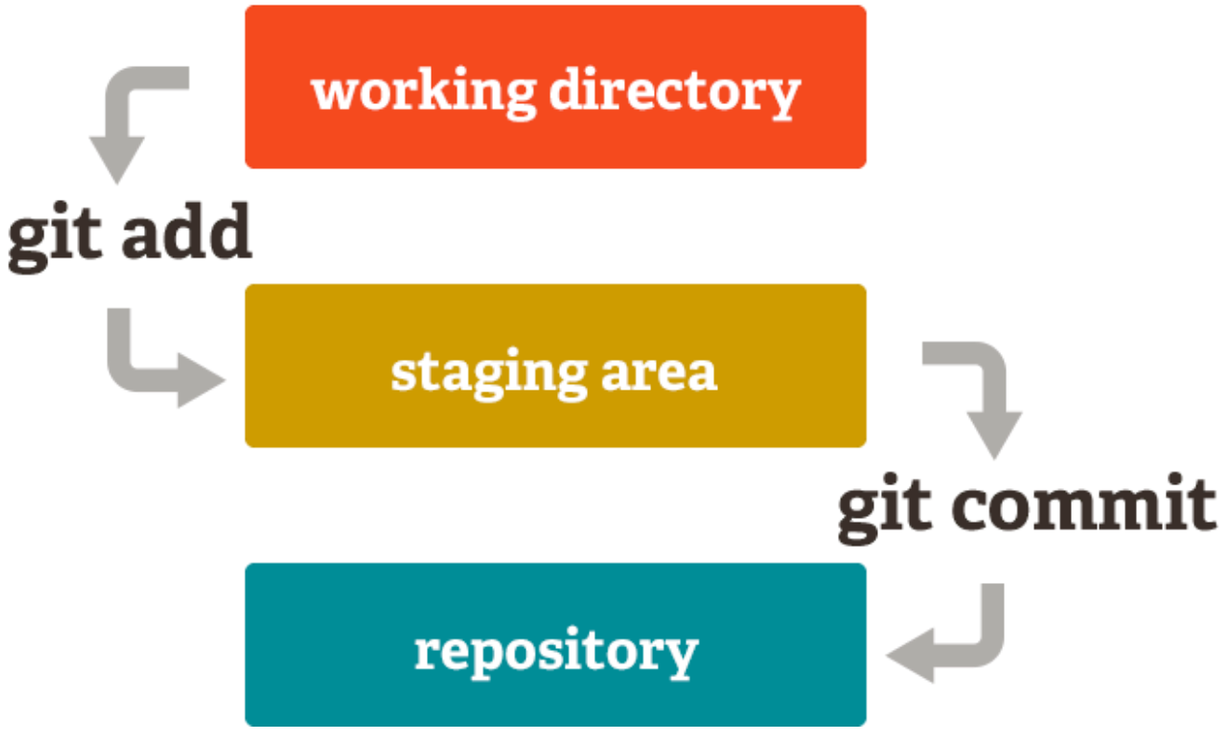
🧐 Significa: *"Ok Git, prendi lo stato temporaneo di staging e fissalo sulla mia linea temporale"*.

Commit:
è l’atto di creare uno stato recuperabile del progetto.

Può essere sia un verbo che un nome: "ho commitato" oppure "quel commit".

GIT - Workflow

Modifiche => Staging area => Commit



GIT - Clients

Git UI / Git CLI

Git è il nome del software che gestisce i file, tiene traccia delle modifiche, etc...
La particolarità è che non ha una vera e propria interfaccia.
Ma allora come si fanno le operazioni di stage, commit, ... ? Con un ***Git client***!

Un Git client è un software che interagisce con Git per fare le operazioni.
Ce ne sono diversi, di diverso tipo:

- **Git-cli**: il client ufficiale, utilizzabile solo da riga di comando (*command line interface*)
- diversi **Git client stand-alone con interfaccia grafica**, utilizzabili come un qualsiasi altro programma (*gitkraken, smartgit, git tower, source tree, ...*)
- **Git client integrati negli IDE**: in tutti è installabile, in alcuni è già installato di default

GIT - Clients

La scelta consigliata

Git client integrato nell'IDE:

nel nostro caso useremo **Antigravity IDE**

https://antigravity.google/product/antigravity-ide



GIT - Revert

Ritorno ad un commit precedente

(operazione avanzata)

Per tornare ad un commit precedente è possibile eseguire una operazione di revert, indicando l'*hash* (o un range di hash) dei/del* commit che si vuole revertare, tornando di fatto alla situazione precedente.

Questa azione non cancella commit, ma fa una copia dello stato precedente del progetto in un nuovo commit, aggiungendolo alla linea temporale.

Revert:
operazione per riportare il repository ad uno stato precedente senza modificare la storia passata

***hash:**
codice univoco identificativo del commit

GIT - Reset

Ritorno ad un commit precedente

(operazione avanzata)

E' possibile anche eseguire una operazione di reset, indicando l'*hash* del commit a cui si vuole ritornare.
Esistono due tipi di reset:

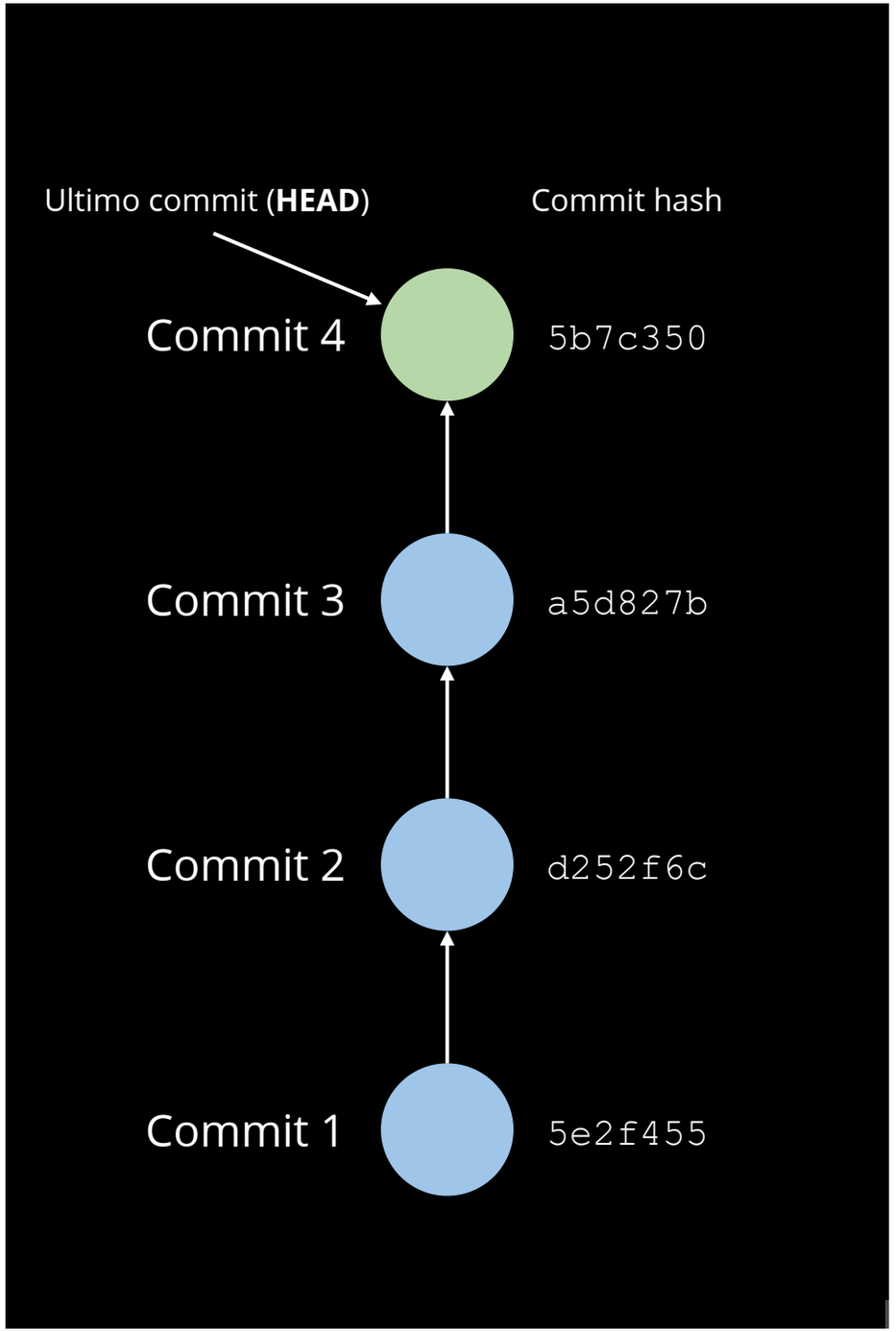
- *Hard Reset*, tutte le modifiche successive al commit vengono eliminate
- *Reset*, tutte le modifiche successive al commit vengono messe in staging

⚠️ In linea generale alterare la linea temporale può portare diversi problemi, è quindi preferibile evitarlo quando possibile.

Reset:
operazione per riportare il repository ad uno stato precedente, cambia la storia passata

GIT - Commits

Come funziona la history dei commit?



GIT - Remote

Salvare la repo sul cloud

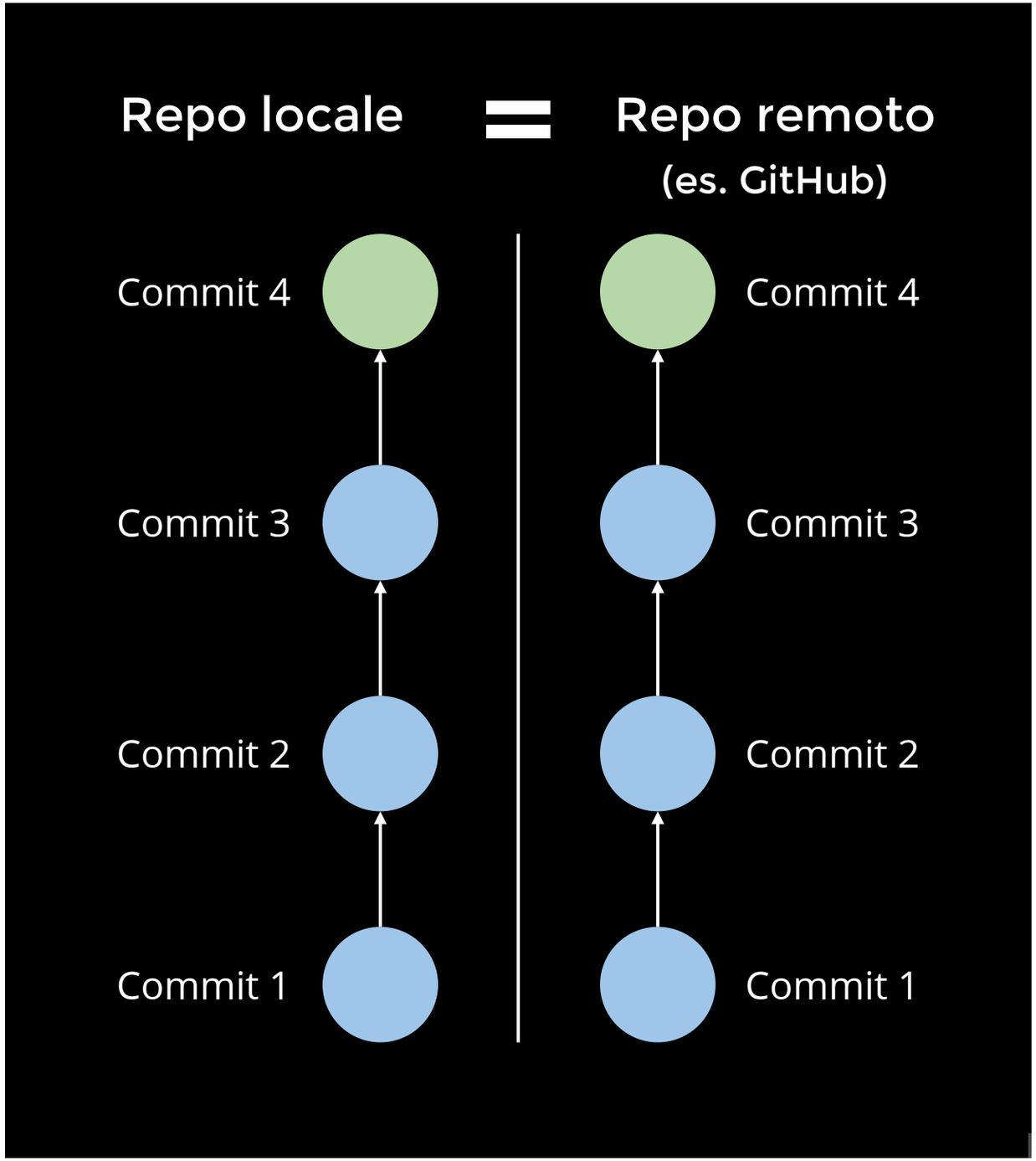
È possibile inviare la copia del nostro repo ad un server esterno, per diversi motivi, tra cui:

- per sicurezza (se il codice fosse solo sul nostro PC e questo si rompesse?)
- per facilitare la condivisione del codice con altre persone

Repository remoto (o remote):
è una copia del repo locale su un server esterno (remoto)

GIT - Remote

Come funziona il repo remoto?



GIT - Push

Sincronizzazione repo - da local a remote

Per far si che il nostro remote abbia una vera utilità dobbiamo caricarci tutte le informazioni del nostro repo locale man mano che ci lavoriamo.

Il push fa proprio questo, quindi caricherà tutto lo stato del nostro repo locale. Sia i file con le modifiche, sia eventuali file nuovi, sia tutte le informazione di tracciamento di Git.

È importante farlo di frequente e non solo a fine giornata.

🧐 Significa: *"Ok Git, prendi il mio repo locale (file, commit, etc.) e porta tutte le novità sul remote per allinearlo"*.

Push:
è l'atto di uploadare il nuovo stato del repo local sul remote.

Può essere sia un verbo che un nome: "ho pushato" oppure "l'ultimo push".


In case of fire
1. git commit
2. git push
3. leave building

GitHub

Il più famoso repository remoto Git

Oltre ad essere ottimo per tenere al sicuro il proprio codice, Github aggiunge funzionalità social per condividere codice, seguire le attività di sviluppo di altri progetti e contribuire.

Inoltre, per i developer, è una sorta di portfolio dei propri progetti (come *Behance* per i creativi) utilizzato anche da HR per fare scouting o valutare i candidati.



Git e GitHub

GitHub diventerà il nostro unico modo di scambiarci progetti e consegnare gli esercizi
(niente più .zip e file via slack, yay!)

Ogni studente dovrà utilizzare Git sul proprio pc
per gestire il proprio codice e sincronizzare il proprio repo con Github

Questo porterà alcuni vantaggi:

- per gli studenti, tutti i vantaggi detti per Git
- per gli insegnanti, la possibilità di accedere al codice dello studente in maniera semplice e di fornire supporto in maniera più veloce



Guida alla sopravvivenza del web developer

Git non fa quello che gli dico!

Git - FAQ

- Posso fare **commit** senza fare uno **stage**? **No!**
  Senza lo stage, Git non sa cosa vuoi conservare nella linea temporale.
- Posso fare un **push** senza un **commit**? **No!**
  Senza un commit Git non sa cosa vuoi inviare alla repository remota.
- Posso dire a Git di **ignorare un file**? **Sì!**
  Git legge il contenuto di un file .gitignore che conterrà la lista di file da ignorare.
- Cosa è un file **.DS_Store**?
  MacOs genera questo file speciale ogni volta che entriamo in una cartella, serve quindi solo al sistema operativo e **possiamo dire a Git di ignorarlo**, vedi il punto precedente.
- **Ho sbagliato a scrivere il nome della repo** su GitHub, cosa posso fare?
  Sul sito Github, nella pagina della nostra repository troviamo la tab **settings**, cambiamo il **nome** e clicchiamo **rename**.

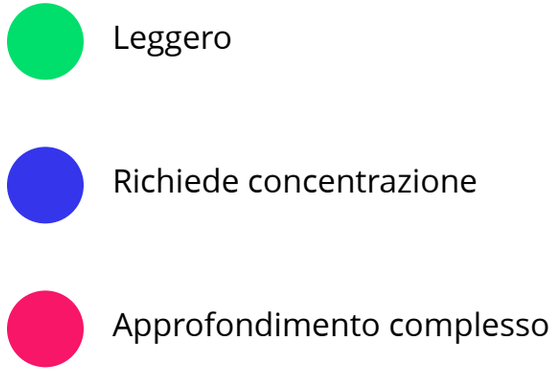
Consigli per la lettura

1 Git Base

https://git-scm.com/book/it/v2

2 Git - La guida tascabile

http://rogerdudler.github.io/git-guide/index.it.html



ESERCITAZIONE